In [62]:
```
# Imports
import os
import sys
import warnings
import numpy as np
import pandas as pd
import pyfixest as pf
import seaborn as sns
import matplotlib.pyplot as plt
import pyreadstat
from statsmodels.nonparametric.smoothers_lowess import lowess
import statsmodels.api as sm
from typing import List
from sklearn.metrics import mean_squared_error, r2_score, log_loss
from stargazer.stargazer import Stargazer
from IPython.display import display, Latex, HTML
from statsmodels.iolib.summary2 import summary_col
pd.options.mode.chained_assignment = None
warnings.filterwarnings("ignore")
```

In [63]:
```
share = pd.read_csv("https://osf.io/download/kbjzp/")
share = share.query("wave==4 & age>=50 & age<=64")

# Some reminder code to help explore data.
share["br015"].unique()
share["mar_stat"].value_counts()
share.describe()
```

Out[63]:

	wave	country	country_mod	int_year	int_month	female	age	eduyears_mod	sph
count	26254.0	26254.000000	26254.000000	26254.000000	26254.000000	26254.000000	26254.000000	26254.000000	26254.0000
mean	4.0	22.593205	349.048640	2011.005028	5.797364	0.562048	57.572903	10.149901	2.9725
std	0.0	8.230092	241.682133	0.236737	2.712384	0.496145	3.903833	6.681391	1.5259
min	4.0	11.000000	40.000000	2010.000000	1.000000	0.000000	50.000000	-15.000000	-15.0000
25%	4.0	16.000000	203.000000	2011.000000	4.000000	0.000000	54.299999	8.000000	2.0000
50%	4.0	20.000000	250.000000	2011.000000	5.000000	1.000000	57.799999	11.000000	3.0000
75%	4.0	32.000000	616.000000	2011.000000	8.000000	1.000000	60.900002	14.000000	4.0000
max	4.0	35.000000	756.000000	2012.000000	12.000000	1.000000	64.000000	25.000000	5.0000

Part 1

1.1

In [64]:

```
# Filter Data
share = share.query('bmi > 0 & mar_stat > 0 & ever_smoked > 0 & smoking > 0 & br015 > 0 & sphus > 0 & eduyears_mod >
share.describe()
```

Out[64]:

	wave	country	country_mod	int_year	int_month	female	age	eduyears_mod	sph
count	23926.0	23926.000000	23926.000000	23926.000000	23926.000000	23926.000000	23926.000000	23926.000000	23926.0000
mean	4.0	22.861364	342.346945	2011.001546	5.824459	0.558597	57.523869	11.432208	3.0305
std	0.0	8.237887	237.699239	0.236567	2.713939	0.496565	3.902150	4.072558	1.0713
min	4.0	11.000000	40.000000	2010.000000	1.000000	0.000000	50.000000	1.000000	1.0000
25%	4.0	16.000000	203.000000	2011.000000	4.000000	0.000000	54.299999	9.000000	2.0000
50%	4.0	23.000000	250.000000	2011.000000	6.000000	1.000000	57.700001	12.000000	3.0000
75%	4.0	32.000000	528.000000	2011.000000	8.000000	1.000000	60.900002	14.000000	4.0000
max	4.0	35.000000	756.000000	2012.000000	12.000000	1.000000	64.000000	25.000000	5.0000

1.2

In [65]:
```
share['exercise'] = ((share["br015"] == 1) | (share["br015"] == 2)).astype(int)
percentExercise = np.sum(share['exercise'])/len(share['exercise'])
percentExercise
```

Out[65]:
```
np.float64(0.5703836830226532)
```

Physical activity even once a week is regular enough to potentially offer health benefits over a more sedentary lifestyle, so both a 1 or 2 in the br015 variable were coded as engaging in exercise. Under this interpretation, 57.04% of the sample would be defined as engaging in exercise.

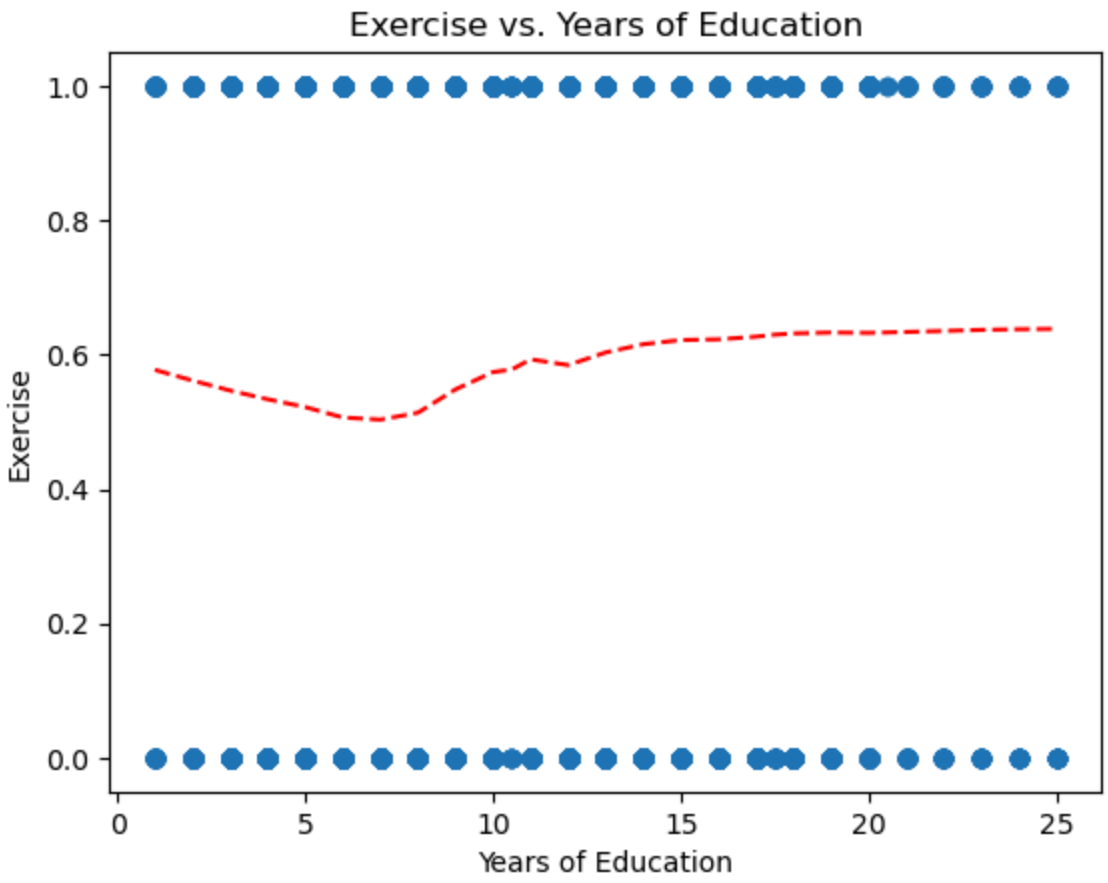
1.3

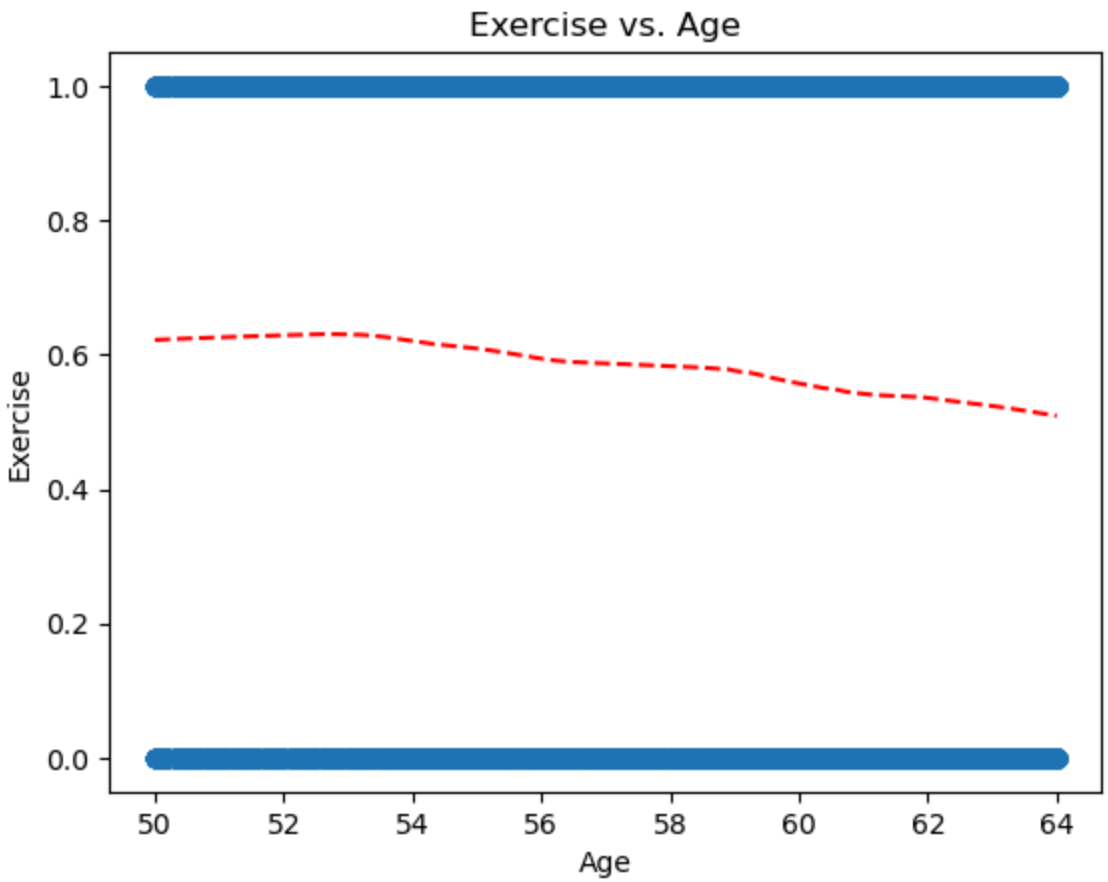
In [66]:
```
share['married'] = ((share["mar_stat"] == 1) | (share["mar_stat"] == 2)).astype(int)
```

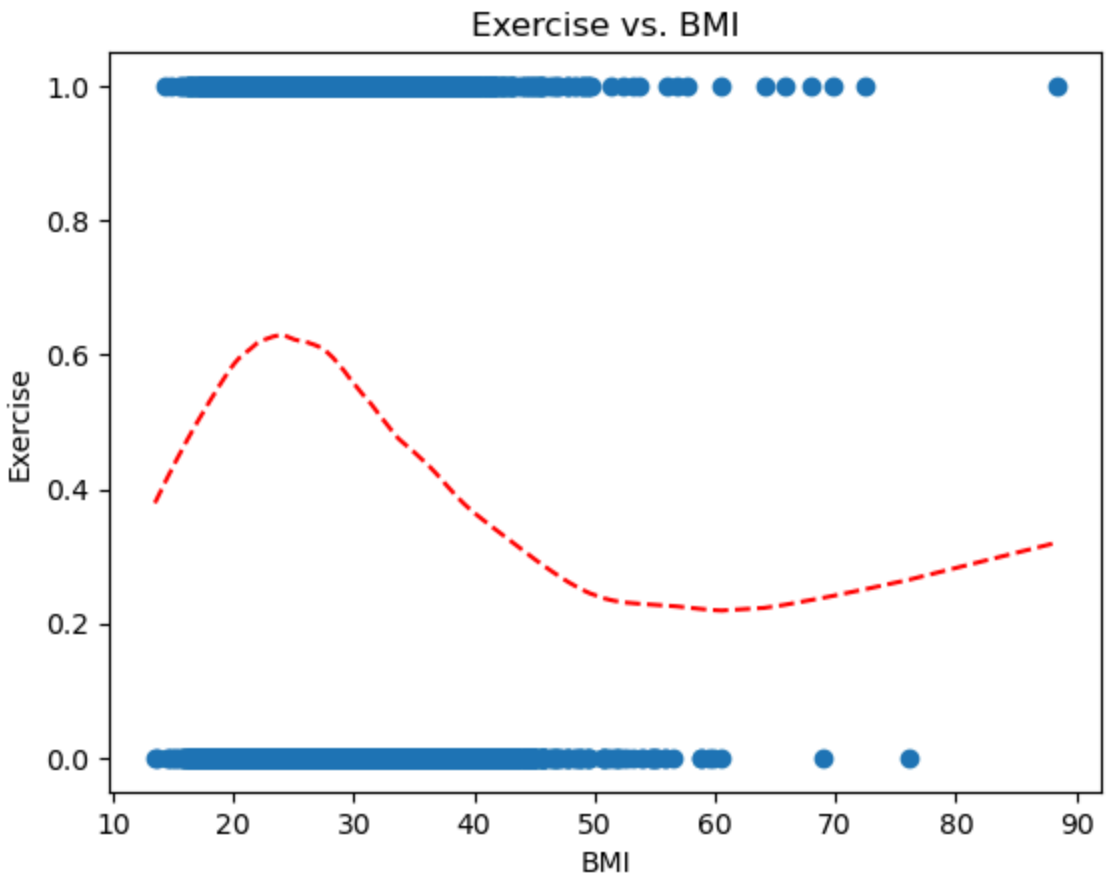
Part 2

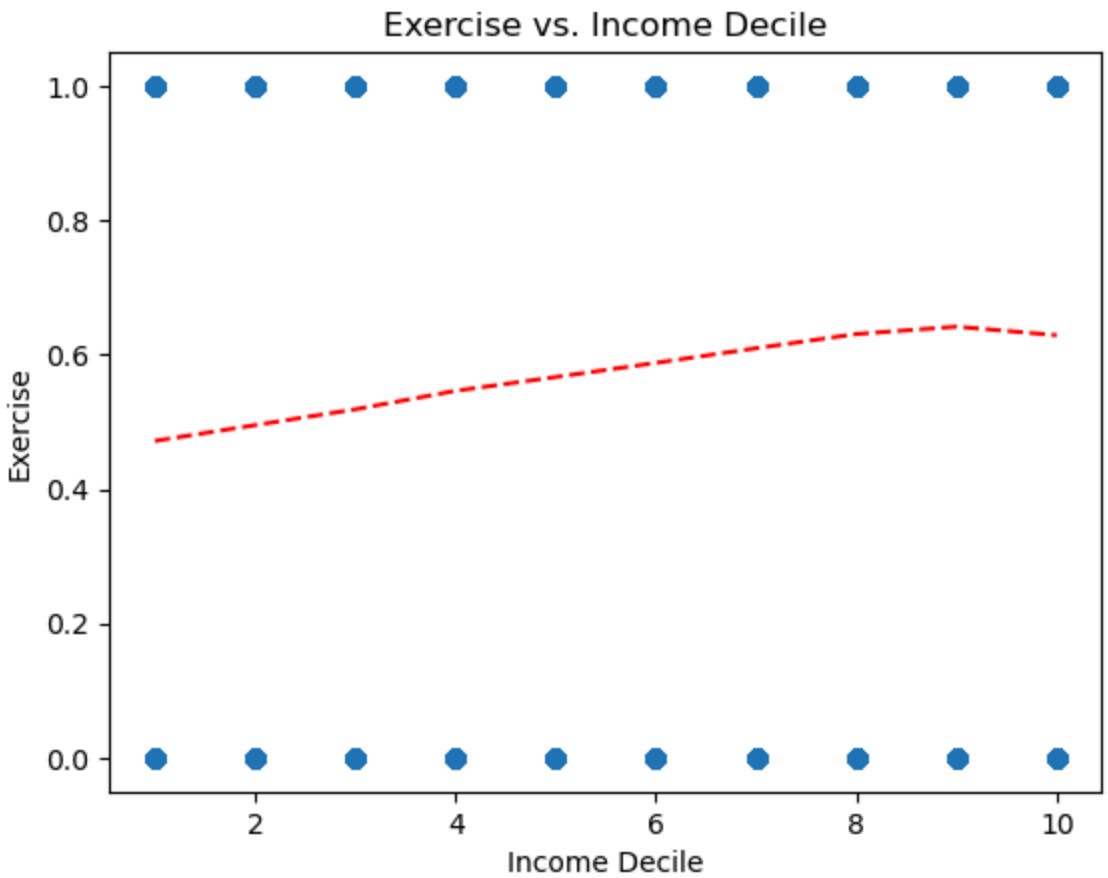
2.1

In [67]:
```
# Lowess Curves to Find Patterns/Splines
plt.figure()
plt.scatter(share['eduyears_mod'],share['exercise'])
lowessfit1 = lowess(share['exercise'],share['eduyears_mod'],frac=1/3.0)
plt.plot(lowessfit1[:,0],lowessfit1[:,1],'r--')
plt.xlabel("Years of Education")
plt.ylabel("Exercise")
plt.title("Exercise vs. Years of Education")

plt.figure()
plt.scatter(share['age'],share['exercise'])
lowessfit2 = lowess(share['exercise'],share['age'],frac=1/3.0)
plt.plot(lowessfit2[:,0],lowessfit2[:,1],'r--')
plt.xlabel("Age")
plt.ylabel("Exercise")
plt.title("Exercise vs. Age")

plt.figure()
plt.scatter(share['bmi'],share['exercise'])
lowessfit3 = lowess(share['exercise'],share['bmi'],frac=1/3.0)
plt.plot(lowessfit3[:,0],lowessfit3[:,1],'r--')
plt.xlabel("BMI")
plt.ylabel("Exercise")
plt.title("Exercise vs. BMI")


plt.figure()
plt.scatter(share['income_pct_w4'],share['exercise'])
lowessfit4 = lowess(share['exercise'],share['income_pct_w4'],frac=1/3.0)
plt.plot(lowessfit4[:,0],lowessfit4[:,1],'r--')
plt.xlabel("Income Decile")
plt.ylabel("Exercise")
plt.title("Exercise vs. Income Decile")

plt.show()
```


Exercise vs. Years of Education
1.0
0.8
0.6
0.4
0.2
0.0
Exercise
0
5
10
15
20
25
Years of Education


Exercise vs. Age
1.0
0.8
0.6
0.4
0.2
0.0
Exercise
50
52
54
56
58
60
62
64
Age


Exercise vs. BMI
Exercise
BMI
1.0
0.8
0.6
0.4
0.2
0.0
10
20
30
40
50
60
70
80
90



2.2

Based on the graphs from 2.1, the splines would be most appropriate for the Years of Education, BMI, and income decile variables. The age lowess curve shows a pretty consistent decreasing trend throughout, so a spline won't be used (which likely can be explained by already using a relatively narrow age range for this sample). For income decile, the spline should be set at 9, which is when the lowess curve changes direction. For the BMI, the knots for the splines will be at 24 and 56. Finally, for the years of education, the knots for the splines will be at 7 and 17. The locations of these knots was based on where the general trend of the Lowess curve appeared to change most substantially, delineating patterns in the data.

2.3

In [68]:
```
# Regression i
x1 = share.loc[:,['female','age']]
x1 = sm.add_constant(x1)
y1 = share['exercise']
model1 = sm.OLS(y1,x1)
res1 = model1.fit(cov_type="HC1")
```

In [69]:
```
# Regression ii
x = share["eduyears_mod"].to_numpy()
share["edu1"] = np.minimum(x, 7)
share["edu2"] = np.minimum(np.maximum(x - 7, 0.0), 17 - 7)
share["edu3"] = np.maximum(x - 17, 0.0)

x = share["bmi"].to_numpy()
share["bmi1"] = np.minimum(x, 24)
share["bmi2"] = np.minimum(np.maximum(x - 24, 0.0), 56 - 24)
share["bmi3"] = np.maximum(x - 56, 0.0)

x = share["income_pct_w4"].to_numpy()
share["inc1"] = np.minimum(x, 9)
share["inc2"] = np.maximum(x - 9, 0.0)


x2 = share.loc[:,['female','age','bmi1','bmi2','bmi3','married','inc1','inc2','edu1','edu2','edu3']]
x2 = sm.add_constant(x2)
y2 = share['exercise']
model2 = sm.OLS(y2,x2)
res2 = model2.fit(cov_type="HC1")
```

In [70]:
```
# Regression iii
cntryDummies = pd.get_dummies(share['country'],prefix='Country ',prefix_sep=' ').astype(int)
x3 = pd.concat([share.loc[:,['female','age','bmi1','bmi2','bmi3','married','inc1','inc2','edu1','edu2','edu3']],cntry
x3 = sm.add_constant(x3)
y3 = share['exercise']
model3 = sm.OLS(y3,x3)
res3 = model3.fit(cov_type="HC1")
```

In [71]:
```
varNames = {'female':'Female', 'age':'Age', 'bmi1':'BMI <= 24', 'bmi2':'24 < BMI <= 56', 'bmi3':'BMI > 56', 'married
            'inc2':'Income Decile > 9', 'edu1': "Years of Education <= 7", 'edu2': '7 < Years of Education <= 17', 'e
```

```
stargazer = Stargazer([res1, res2, res3])
stargazer.covariate_order(['female','age','married', 'bmi1','bmi2', 'bmi3', 'edu1','edu2','edu3', 'inc1','inc2'] + cr
stargazer.rename_covariates(varNames)
stargazer.custom_columns(['Model 1','Model 2','Model 3'])
stargazer.dependent_variable_name('Exercise')
#display(Latex(stargazer.render_latex()))
display(HTML(stargazer.render_html()))
```

			Dependent variable: Exercise
	Model 1	Model 2	Model 3
	(1)	(2)	(3)
Female	-0.097***	-0.093***	-0.093***
	(0.006)	(0.006)	(0.006)
Age	-0.009***	-0.007***	-0.008***
	(0.001)	(0.001)	(0.001)
Married		-0.007	-0.001
		(0.008)	(0.008)
BMI <= 24		0.011***	0.011***
		(0.003)	(0.003)
24 < BMI <= 56		-0.013***	-0.013***
		(0.001)	(0.001)
BMI > 56		0.032***	0.029***
		(0.009)	(0.009)
Years of Education <= 7		-0.003	-0.001
		(0.003)	(0.004)
7 < Years of Education <= 17		0.006***	0.002*
		(0.001)	(0.001)
Years of Education > 17		-0.007	-0.005
		(0.005)	(0.004)

Income Decile <= 9	0.016***	0.016***
	(0.001)	(0.001)
Income Decile > 9	-0.030***	-0.027***
	(0.010)	(0.010)
Country 11		0.102***
		(0.011)
Country 12		0.202***
		(0.020)
Country 13		0.165***
		(0.021)
Country 14		0.213***
		(0.014)
Country 15		-0.115***
		(0.015)
Country 16		-0.032**
		(0.014)
Country 17		-0.018
		(0.011)
Country 18		0.071***
		(0.015)
Country 20		0.082***
		(0.012)

Country 23			-0.002
			(0.011)
Country 28			0.002
			(0.011)
Country 29			-0.047**
			(0.019)
Country 32			-0.011
			(0.013)
Country 33			-0.094***
			(0.018)
Country 34			0.118***
			(0.014)
Country 35			0.112***
			(0.010)
Constant	1.125***	0.745***	0.748***
	(0.047)	(0.080)	(0.076)
Observations	23926	23926	23926
R^2	0.014	0.035	0.062
Adjusted R^2	0.014	0.034	0.061
Residual Std. Error	0.492 (df=23923)	0.486 (df=23914)	0.480 (df=23899)
F Statistic	172.259*** (df=2; 23923)	83.534*** (df=11; 23914)	1364.041*** (df=26; 23899)

Note:	*p<0.1; **p<0.05; ***p<0.01

2.4

In both models 2 and 3, the coefficient on female is -0.093, significant at 99%, which means that, holding all other variables constant, women are, on average, 9.3% less likely to exercise than men. The age variable has a coefficient of -0.007 (or -0.008 in model 3), which means that, on average, holding all other factors constant, for every year older a person is, they are 0.7% (or 0.8%) less likely to exercise. Finally, for the BMI spline, the coefficients for the 3 intervals in model 2 are 0.011, -0.013, and 0.032, all significant at 99%. These mean that, holding all other variables constant, on average, every point of BMI up to 24 increases the likelihood of exercising by 1.1%. For every additional point of BMI between 24 and 56, the average likelihood of exercising goes down by 1.3%. Finally, for those with BMI above 56, every additional point of BMI translates to, on average, an additional 3.2% higher likelihood of exercising.

2.5

The first interesting insight, per the coefficient on female, is that men are on average 9.3% more likely to exercise. Given the slightly looser definition of exercise used in this analysis, perhaps this could partially be explained by men dominating the labor force in physically intensive fields, where work such as construction may fall under the definition of exercise.

A second interesting observation comes from BMI. For BMIs up to 24, these are correlated with a higher likelihood of exercising, while higher BMIs between 24 and 56 correlate with a lower likelihood of exercising. Given that 25 is the BMI typically considered the cutoff for being overweight, it is rather unsurprising that those that are overweight or obese are less likely to exercise (given that obesity tends to be caused in part by a more sedentary lifestyle). However, the increasing likelihood for those at extremely high BMIs above 56 is difficult to explain.

Third, the income decile yields interesting results. In general, the more a person makes, the more likely they are to exercise, which could perhaps be explained by wealthier individuals generally having more disposable income to spend on athletic activities and have more free time to dedicate to such activities. The dropoff at those in the highest decile is intriguing though.

2.6

In [72]:
```
x4 = share.loc[:,['female','age','bmi','married','income_pct_w4','eduyears_mod']]
x4 = sm.add_constant(x4)
y4 = share['exercise']
```

```
model4 = sm.OLS(y4,x4)
res4 = model4.fit(cov_type="HC1")

varNames2 = {'female':'Female', 'age':'Age', 'bmi1':'BMI <= 24', 'bmi2':'24 < BMI <= 56', 'bmi3':'BMI > 56', 'marrie
            'inc2':'Income Decile > 9', 'edu1': "Years of Education <= 7", 'edu2': '7 < Years of Education <= 17', '
            'bmi':'BMI', 'eduyears_mod': 'Years of Education', 'income_pct_w4': 'Income Decile'}

stargazer2 = Stargazer([res2, res4])
stargazer2.covariate_order(['female','age','married', 'bmi', 'bmi1','bmi2', 'bmi3','eduyears_mod', 'edu1','edu2','edu
stargazer2.rename_covariates(varNames2)
stargazer2.custom_columns(['Model 2','Model 4'])
stargazer2.dependent_variable_name('Exercise')
#display(Latex(stargazer2.render_latex()))
display(HTML(stargazer2.render_html()))
```

	Dependent variable: Exercise	
	Model 2	Model 4
	(1)	(2)
Female	-0.093***	-0.099***
	(0.006)	(0.006)
Age	-0.007***	-0.007***
	(0.001)	(0.001)
Married	-0.007	-0.002
	(0.008)	(0.008)
BMI		-0.009***
		(0.001)
BMI <= 24	0.011***	
	(0.003)	
24 < BMI <= 56	-0.013***	
	(0.001)	
BMI > 56	0.032***	
	(0.009)	
Years of Education		0.003***
		(0.001)
Years of Education <= 7	-0.003	
	(0.003)	

7 < Years of Education <= 17	0.006***	
	(0.001)	
Years of Education > 17	-0.007	
	(0.005)	
Income Decile		0.014***
		(0.001)
Income Decile <= 9	0.016***	
	(0.001)	
Income Decile > 9	-0.030***	
	(0.010)	
Constant	0.745***	1.152***
	(0.080)	(0.051)
Observations	23926	23926
R^2	0.035	0.031
Adjusted R^2	0.034	0.031
Residual Std. Error	0.486 (df=23914)	0.487 (df=23919)
F Statistic	83.534*** (df=11; 23914)	131.280*** (df=6; 23919)
Note:		*p<0.1; **p<0.05; ***p<0.01

Using the splines, especially for education and BMI, was appropriate. The model slightly better at predicting exercise, with $R^2 = 0.035$ with splines, compared to $R^2 = 0.031$ without them. Additionally, for years of education, the spline shows that years of education is only a significant contributor in the middle interval, while years of education below 7 or above 17 are not significant - not using the spline would lose this nuance. The effect for BMI is similar, and not using a spline would mean not capturing the first

interval where BMI is correlated with a higher average likelihood of exercising. Finally, because the spline for income decile was so limited, it did not make a substantial effect on the regression (with coefficients of 0.016 for most of the spline or 0.014 for no spline).

Part 3

3.1

In [73]:
```
model5 = sm.Logit(y2,x2)
res5 = model5.fit(cov_type="HC1")
effect5 = res5.get_margeff()

model6 = sm.Probit(y2,x2)
res6 = model6.fit(cov_type="HC1")
effect6 = res6.get_margeff()


stargazer3 = Stargazer([res2, res5, res6])
stargazer3.covariate_order(['female','age','married', 'bmi1','bmi2', 'bmi3', 'edu1','edu2','edu3', 'inc1','inc2','cor
stargazer3.rename_covariates(varNames)
stargazer3.custom_columns(['LPM','Logit','Probit'])
stargazer3.dependent_variable_name('Exercise')
#display(Latex(stargazer3.render_latex()))
display(HTML(stargazer3.render_html()))
```

```
Optimization terminated successfully.
         Current function value: 0.665647
         Iterations 5
Optimization terminated successfully.
         Current function value: 0.665641
         Iterations 5
```

	Dependent variable: Exercise		
	LPM	Logit	Probit
	(1)	(2)	(3)
Female	-0.093***	-0.390***	-0.242***
	(0.006)	(0.027)	(0.017)
Age	-0.007***	-0.031***	-0.019***
	(0.001)	(0.003)	(0.002)
Married	-0.007	-0.028	-0.017
	(0.008)	(0.033)	(0.020)
BMI <= 24	0.011***	0.046***	0.028***
	(0.003)	(0.011)	(0.007)
24 < BMI <= 56	-0.013***	-0.054***	-0.034***
	(0.001)	(0.004)	(0.002)
BMI > 56	0.032***	0.160**	0.097***
	(0.009)	(0.068)	(0.037)
Years of Education <= 7	-0.003	-0.013	-0.008
	(0.003)	(0.014)	(0.009)
7 < Years of Education <= 17	0.006***	0.027***	0.017***
	(0.001)	(0.005)	(0.003)
Years of Education > 17	-0.007	-0.031	-0.019
	(0.005)	(0.019)	(0.012)

Income Decile <= 9	0.016***	0.068***	0.042***
	(0.001)	(0.006)	(0.004)
Income Decile > 9	-0.030***	-0.127***	-0.079***
	(0.010)	(0.044)	(0.027)
Constant	0.745***	1.022***	0.635***
	(0.080)	(0.333)	(0.206)
Observations	23926	23926	23926
R^2	0.035		
Adjusted R^2	0.034		
Pseudo R^2		0.026	0.026
Residual Std. Error	0.486 (df=23914)		
F Statistic	83.534*** (df=11; 23914)		
Note:	*p<0.1; **p<0.05; ***p<0.01		

In [74]:
```
effect5.summary()
```

Out[74]:

Logit Marginal Effects

Dep. Variable:	exercise
Method:	dydx
At:	overall

	dy/dx	std err	z	P>|z|	[0.025	0.975]
female	-0.0922	0.006	-14.464	0.000	-0.105	-0.080
age	-0.0074	0.001	-9.123	0.000	-0.009	-0.006
bmi1	0.0108	0.003	4.000	0.000	0.006	0.016
bmi2	-0.0128	0.001	-15.380	0.000	-0.014	-0.011
bmi3	0.0379	0.016	2.367	0.018	0.007	0.069
married	-0.0067	0.008	-0.863	0.388	-0.022	0.009
inc1	0.0161	0.001	11.842	0.000	0.013	0.019
inc2	-0.0300	0.010	-2.888	0.004	-0.050	-0.010
edu1	-0.0030	0.003	-0.871	0.384	-0.010	0.004
edu2	0.0065	0.001	5.078	0.000	0.004	0.009
edu3	-0.0073	0.005	-1.590	0.112	-0.016	0.002

In [75]:

```
effect6.summary()
```

Out[75]:

Probit Marginal Effects

Dep. Variable:	exercise
Method:	dydx
At:	overall

	dy/dx	std err	z	P>|z|	[0.025	0.975]
female	-0.0925	0.006	-14.491	0.000	-0.105	-0.080
age	-0.0074	0.001	-9.134	0.000	-0.009	-0.006
bmi1	0.0108	0.003	3.992	0.000	0.005	0.016
bmi2	-0.0128	0.001	-15.445	0.000	-0.014	-0.011
bmi3	0.0372	0.014	2.635	0.008	0.010	0.065
married	-0.0065	0.008	-0.829	0.407	-0.022	0.009
inc1	0.0161	0.001	11.851	0.000	0.013	0.019
inc2	-0.0301	0.010	-2.914	0.004	-0.050	-0.010
edu1	-0.0029	0.003	-0.846	0.398	-0.010	0.004
edu2	0.0064	0.001	5.055	0.000	0.004	0.009
edu3	-0.0074	0.005	-1.613	0.107	-0.016	0.002

3.2

In [76]:

```
share['pred_lpm'] = res2.predict(x2)
share['pred_logit'] = res5.predict(x2)
share['pred_probit'] = res6.predict(x2)


preds = {
    "LPM": share["pred_lpm"].astype(float),
    "Logit": share["pred_logit"].astype(float),
    "Probit": share["pred_probit"].astype(float),
}
```

```
eps = 1e-15
preds_clip = {k: v.clip(eps, 1 - eps) for k, v in preds.items()}
y = share["exercise"].astype(float)

metrics = pd.DataFrame(
    {
        "R-squared": {
            "LPM": float(res2.rsquared),
            "Logit": r2_score(y, preds["Logit"]),
            "Probit": r2_score(y, preds["Probit"]),
        },
        "Brier-score": {k: mean_squared_error(y, v) for k, v in preds.items()},
        "Pseudo R-squared": {
            "LPM": np.nan,
            "Logit": float(res5.prsquared),
            "Probit": float(res6.prsquared),
        },
        "Log-loss": {k: log_loss(y, v) for k, v in preds_clip.items()},
    }
).T.round(5)

metrics
```

Out[76]:

	LPM	Logit	Probit
R-squared	0.03467	0.03467	0.03468
Brier-score	0.23655	0.23655	0.23655
Pseudo R-squared	NaN	0.02570	0.02571
Log-loss	0.66564	0.66565	0.66564

The three models - LPM, Logit, and Probit - in this case yield nearly identical results. The marginal effects for Models 5 (Logit) and 6 (Probit) are nearly the same at 3-4 decimal places, and very closely match the coefficients of Model 2 at 3 decimal places. Under the other metrics used to evaluate the models, there is also almost nothing to distinguish them. They have the same Brier score to 5 decimal places. The log-loss score for the Logit model is 0.00001 higher than the other two, and the pseudo-R^2 of the Probit model is 0.00001 higher than that of the Logit model. So under these two measures, the Probit model is technically the best, but the differences are extremely marginal. It has the added advantage of not allowing a result over 1 like the LPM model might.